

Renewable Asset Performance Triage

Data Scientist Take-Home — Proof of Concept & Design Memo

Scope: solar v1 · A small, leakage-aware baseline + rule-based attribution + financial ranking

Executive summary

The problem. Today, whether a renewable asset is underperforming, and why, is answered across disconnected dashboards and spreadsheets. Revenue quietly leaks while the signal is split across teams.

What the v1 delivers. Every morning, one ranked list of the solar assets most likely losing revenue, each with a probable cause (outage, curtailment, degradation, or a data-quality blind spot) and a concrete next step for the operator. Issues are ranked by estimated lost revenue in \$, not raw MWh, so attention follows money.

What it will not do (yet). It does not cover wind or batteries, does not forecast future generation, and does not replace engineering root-cause analysis, it gives a probable cause and points a human to the right place. It is a daily batch product, not a real-time alarm.

Main risk. Data quality. A frozen irradiance sensor can masquerade as underperformance; the design treats input integrity as a first-class, separately-surfaced flag rather than a silent failure. Alert fatigue from false positives is the second risk, managed with a persistence test that requires a sustained shortfall before flagging, a materiality floor in \$, and an operator feedback loop.

What we need from stakeholders. Read access to the listed data sources, the asset-to-contract mapping, and roughly 30 minutes per week of an operations SME to confirm or reject flags, that feedback is the fuel that improves the system and, later, trains a model.

Part 1. Problem decomposition & POC scope

Problem statement

Each morning, give the solar operations team a **ranked list of assets most likely losing revenue to an actionable cause**, with the probable cause and the next inspection step. Decision supported: **what does an operator inspect first today?** Cadence: daily. Users: operators and asset managers.

Scope boundaries (deliberately out of v1)

- **Solar only.** The expected-generation model for solar (irradiance → power) is the cleanest and most validatable; wind power curves and battery dispatch have different baseline logic and are deferred.
- **Daily batch, not real-time.** Minute-level latency does not change the asset manager's morning decision; streaming multiplies infra cost without proportional value in v1.
- **Probable cause, not root cause.** We narrow to a likely category and route a human; we do not auto-diagnose the specific failed component.
- No forecasting and no maintenance optimization in v1. The system does not do full causal decomposition either, it reports the dominant cause and, when signals conflict, surfaces the secondary one, with the consistency score reflecting the ambiguity.

Success criteria

- **Analytical:** the canonical fault types are detected with correct attribution, including a mixed-cause asset where the diagnostic-consistency score correctly drops below 1.0, and a regional weather event correctly separated from asset-specific faults (peers dipping together); low false-positive rate on resource-driven low output (we must not flag a cloudy day).
- **Business / user:** operators open the list each morning (adoption); time-to-detect on outages drops; the SME agrees the ranking would change what they inspect first.

Key assumptions

- D-1 data is available by morning; telemetry has gaps and occasional frozen sensors; maintenance notes are inconsistent free-text; asset-to-contract mapping exists but is messy; users are semi-technical and will not read code.

Part 2. Analytical / ML approach

Baseline (the core decision). Expected generation is a Performance-Ratio decomposition: a physics term, $\text{capacity} \times (\text{POA irradiance} / 1000) \times \text{temperature de-rate}$, multiplied by PR_baseline, the asset's own healthy median PR. The physics needs no training data;

PR_baseline empirically absorbs wiring losses, inverter efficiency, soiling, and undocumented install geometry. Because PR is dimensionless, residuals are comparable across a 5 MW and a 200 MW plant.

Leakage control (where most candidates slip). PR_baseline is learned only on healthy hours, daylight, available, not curtailed, plausible PR, and only from the first 30 days, so a recent fault can never teach the baseline to accept itself. I used a median, not a mean.

Anomaly detection. A raw dip is observed PR below the asset's own 10th-percentile band with real sun. Because a p10 band crosses ~10% of healthy hours by construction, a dip only counts if it **persists** across at least 3 of the last 4 daytime hours, real faults span hours, while p10 noise is isolated. Physically impossible PR flags immediately as a data-quality tell. The trade-off is a short detection latency (a few daytime hours of evidence before flagging), acceptable for a daily product.

Cause attribution. An ordered rule tree over the auxiliary signals, check the data input first (don't blame a plant when the sensor is broken), then curtailment flag, then availability/inverter status for outage, then a peer-group test that separates regional weather (same-region assets dipping together) from an asset-specific fault, and finally escalate. Degradation is a slow signal detected from the multi-week PR trend. Weather/resource that is simply low but normal needs no rule at all: the baseline normalizes by measured irradiance, so a cloudy day produces a normal PR and never flags, the explicit "Resource/Weather" label is reserved for a real shortfall that peers share.

On the consistency score. When an asset's flagged hours split across causes, the dominant one is reported with a **diagnostic-consistency score** below 1.0 and the runner-up surfaced as a secondary cause to check. This score is the share of flagged hours agreeing with the dominant cause, it is a measure of diagnostic agreement, **not a calibrated probability**. True calibration (where a 0.7 means a 70% hit rate) would require operator-confirmed labels accumulated through the feedback loop, which is a v2 deliverable.

Prioritization. Lost MWh $\times$ the contract-relevant price (fixed PPA vs. volatile merchant). Data-quality issues are surfaced separately with unknown \$ impact rather than buried at \$0, because a blind asset is itself a priority. Resource/Weather events are non-actionable, you cannot fix the weather, so they are pulled out of the action list and collapsed into a one-line regional summary, which stops a regional cloud event from generating dozens of redundant alerts and field dispatches.

Human-in-the-loop. Each flag carries confirm / reject / re-label. That feedback tunes thresholds, measures attribution accuracy over time, and accumulates labels for a v2 model.

Trade-off: rules / PR baseline vs. gradient-boosted model

A LightGBM model (generation ~ irradiance, temp, hour, season) would win a few points of accuracy in complex regimes but becomes a black box, needs more per-asset history, and can't explain **why** it expected a number, which kills operator trust and the feedback loop. Recommendation: ship the interpretable PR baseline in v1; introduce ML in v2 only after operators have confirmed enough labels, and even then as a refinement to the anomaly band, not an opaque replacement. The v1 bottleneck is adoption, not the last 3 points of accuracy.

Trade-off 2: Batch daily workflow vs. near-real-time alerts

The alternative to a daily batch job is a streaming pipeline that evaluates each asset continuously and fires alerts within minutes of a deviation. Near-real-time would reduce time-to-detection for outages and could theoretically recover more revenue per event. However, for the specific decision this system supports, a morning triage that tells operators what to inspect today, minute-level latency does not change the outcome. An outage detected at 6:05 AM vs. 6:50 AM produces the same field dispatch. Streaming infrastructure (Kafka, Flink, stateful consumers) multiplies engineering complexity and operational cost without proportional value in v1. The correct moment to revisit this trade-off is when operators identify a category of fault, for example, a cascading inverter failure with very high per-hour cost, where intra-day detection materially changes the response. Until that threshold is identified with data, batch is the defensible choice: simpler to build, simpler to monitor, and aligned with the daily decision cadence of the operations team.

Part 3. Product & system design

AES Renewable Asset Triage

End-to-end production pipeline — from raw asset signals to ranked operator triage, with a daily analytical core and a human-in-the-loop feedback path.

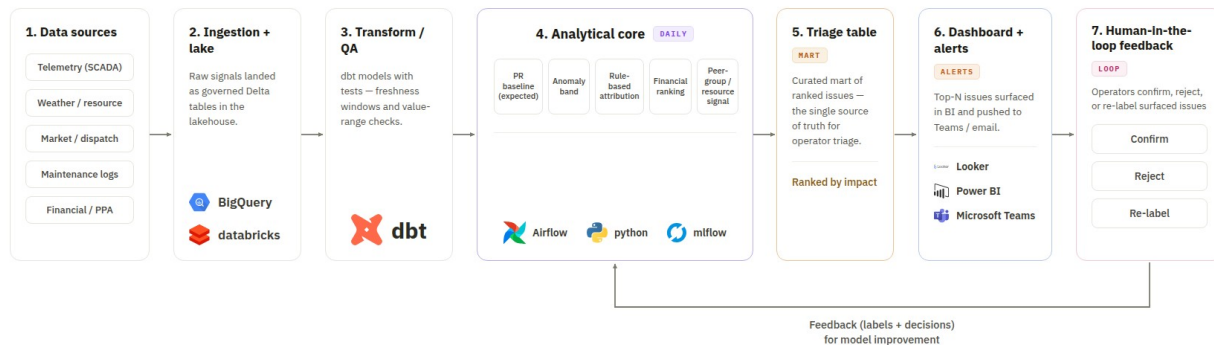


Figure 1: End-to-end architecture (v1).

Technology choices

- **Ingestion + lake:** BigQuery / Databricks Delta, cheap to land raw telemetry, weather, market, and financial feeds.
- **Transform + QA:** dbt, versioned, testable models; freshness and range tests are the first line of data-quality defense.
- **Analytical core:** Python job orchestrated by Airflow, daily; registered in MLflow once the baseline is versioned.
- **Serving:** a curated triage mart feeding Looker / Power BI, with top-N pushed to Teams/email.
- **Why:** every piece already exists in-house, is auditable, and a daily batch matches the morning-triage decision without streaming cost.

Data quality risks & detection

- Frozen/stuck telemetry (zero variance), gaps, cross-source timestamp misalignment, and, most dangerous, wrong asset-to-contract mapping that poisons the \$ ranking. Detection: dbt freshness/null/range tests, zero-variance checks, and capacity-vs-generation reconciliation.

Monitoring

- Daily alert volume, false-positive rate via operator feedback, seasonal data drift (irradiance/price distributions shifting breaks a static baseline), attribution accuracy over time, and adoption (how many operators open the list).

Security & governance

- Access scoped by region/portfolio; audit log of who confirmed/rejected each flag (a decision trail); versioned baselines so any past day's list is reproducible.

POC output. ranked issue list

A solar fleet of 30 assets was simulated hourly for 45 days, with **six ground-truth scenarios injected**: four single-cause faults, one deliberately mixed-cause asset (curtailment on day 1, partial outage on day 2), and one regional weather event hitting every asset in one region for an afternoon. The pipeline recovered all six: the four single-cause faults at consistency 1.0, the mixed asset on

Outage at 0.55 (the consistency score doing its job), and the regional event correctly collapsed into a single non-actionable line rather than nine separate fault alerts. The persistence test left the false-positive tail at zero. The table shows the morning triage over the last 48 hours; the chart ranks performance issues by financial impact.

Asset	Flag	Probable cause	Consist.	Lost MWh (48h)	Est. lost rev. (\$)	Contract
SOLAR_019	Data quality	Sensor/Data issue	1.00	n/a	n/a (blind)	PPA
SOLAR_005	Performance	Outage	1.00	262.6	22,465	Merchant
SOLAR_012	Performance	Degradation	1.00	239.2	18,685	Merchant
SOLAR_027	Performance	Outage (mixed)	0.55	97.2	7,500	Merchant
SOLAR_023	Performance	Curtailment / Market	1.00	58.8	2,775	PPA

Separately, 9 assets in the Southwest region were attributed Resource/Weather (a shared regional event, ~735 MWh) and excluded from the action list as non-actionable — one informational line instead of nine false fault alerts.

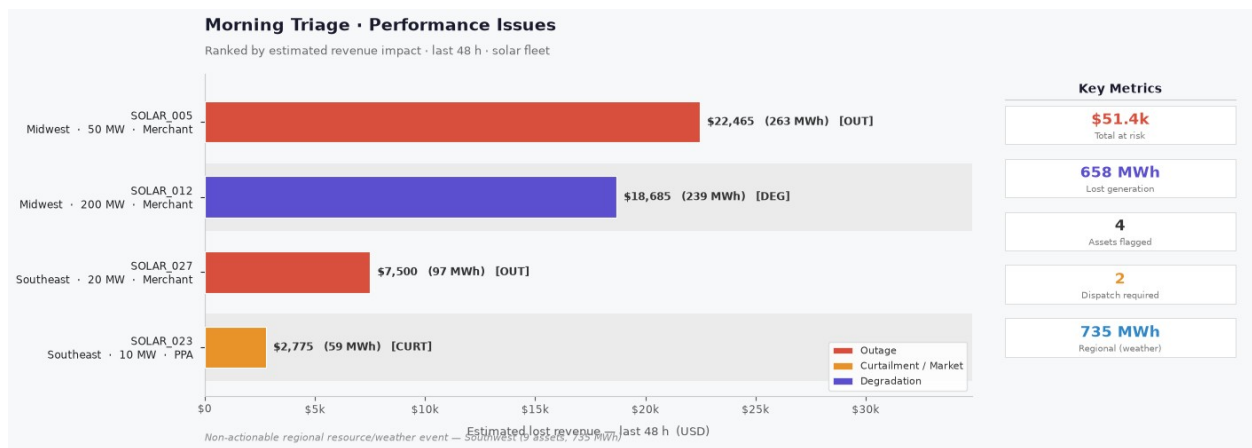


Figure 2: Daily triage, performance issues ranked by estimated lost revenue.



Figure 3: Expected (PR baseline) vs. actual for each injected fault, including the mixed-cause asset (SOLAR_027); red points are flagged hours. Note the flat 'expected' line for the frozen-sensor asset — the data-quality signature.

Reading the flagged assets. SOLAR_005 (Outage): over the last two days, observed PR dropped abruptly below the asset's healthy band while irradiance stayed normal; availability fell to 52% and inverter status read 'fault'. The rule tree clears data-quality and curtailment, then attributes Outage. Estimated impact ~263 MWh ≈ \$22.5k. Next step: dispatch a field tech and pull inverter/string fault logs. SOLAR_027 (mixed) shows the consistency score earning its place: its flagged hours split between curtailment (day 1) and a partial outage (day 2), so it lands on Outage at only 0.55 consistency, with curtailment surfaced as a secondary cause. The Southwest resource event shows the opposite move, nine assets dipping together is read as weather, collapsed to one non-actionable note so no crew is dispatched for the sky. The persistence requirement removed the false-positive tail entirely, the healthy assets correctly produce no flags, trading a few hours of detection latency for clean, trustworthy alerts.

Part 4. Communication & leadership

Team implementation plan (4–6 weeks; 1 DE, 1 DS, 1 SME)

1. **Weeks 1–2:** DE stands up ingestion + dbt for one real solar feed; DS prototypes the PR baseline on synthetic data; SME defines what counts as actionable underperformance.
2. **Weeks 3–4:** DS implements attribution + ranking; DE productionizes the daily Airflow job; SME validates the first flags (the feedback loop is born here).
3. **Weeks 5–6:** minimal dashboard, hardening, and the monitoring baseline (alert volume, false-positive rate).

Guardrails (move fast without an unmaintainable science project)

- Everything in git from day one; no orphan notebook in production. dbt tests required to merge. One success metric agreed with the SME before coding. Weekly review with operators so the product stays anchored to a real decision.

Part 5. Open-ended extension

A GenAI layer over maintenance free-text. The maintenance logs are unstructured free-text that no one mines today. An LLM layer could (a) extract structured cause/severity/equipment from each work order, and (b) generate the operator-facing 'why this was flagged and what to check next' by correlating today's flag with the asset's work-order history.

Why it matters: it turns the static 'next step' column into a context-aware recommendation. **Data needed:** the work-order history plus the POC's flags. **When NOT to pursue:** if the logs are too sparse/inconsistent to give signal, or if hallucinated recommendations could send a crew to inspect the wrong thing, in an operations/safety context that has real cost, so it stays strictly human-in-the-loop and only suggests, never decides.

Note on AI / tool usage & validation

- **Where AI was used:** to accelerate scaffolding of the synthetic-data generator, the plotting code, and structuring this memo.
- **How it was validated:** the analytical logic was verified empirically, six ground-truth scenarios were injected (four single-cause faults, one mixed-cause asset, and one regional weather event) and the pipeline was checked to recover each with the correct attribution (see the validation cell in the notebook; all six pass and the false-positive tail is zero). Baseline leakage was checked by confirming PR_baseline is learned only on pre-fault healthy hours. All numbers in this memo come from the executed notebook, not from the model's assertion.
- **What I would trust least in production:** the rule-based attribution at the boundaries (e.g., simultaneous curtailment + partial outage) and the irradiance sensor itself, both are explicitly monitored.